

LAPORAN PROJECT CLOUD COMPUTING

[RKK401] KOMPUTASI AWAN

**Kalkulator Cerdas Prediksi Harga Properti AI Menggunakan Arsitektur
Event-Driven Microservices**

Kelas: RK – A1



Oleh:

NATHANAEL INDRA R P

(163231004)

PROGRAM STUDI TEKNIK ROBOTIKA DAN KECERDASAN BUATAN

FAKULTAS TEKNOLOGI MAJU DAN MULTIDISIPLIN

UNIVERSITAS AIRLANGGA

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Di era digital saat ini, analisis data berskala besar dan penerapan *Machine Learning* (ML) membutuhkan sumber daya komputasi yang masif. Prediksi harga properti melibatkan banyak variabel kompleks yang sulit dihitung secara manual, mulai dari luas bangunan, jumlah kamar, hingga tren pasar yang dinamis. Diperlukan sebuah sistem cerdas yang mampu memprediksi harga secara real-time berdasarkan data historis.

Cloud computing telah muncul sebagai solusi fundamental untuk mengatasi keterbatasan sumber daya komputasi lokal. Erl, Mahmood, dan Puttini mendefinisikan *cloud computing* sebagai model yang memungkinkan akses jaringan yang nyaman dan sesuai kebutuhan ke kumpulan sumber daya komputasi yang dapat dikonfigurasi dan dirilis dengan cepat dengan upaya manajemen minimal [1].

Proyek ini mengembangkan sistem “Kalkulator Cerdas Prediksi Harga Properti AI” berbasis antarmuka web dengan arsitektur *Event-Driven Microservices*. Beban komputasi berat untuk inferensi kecerdasan buatan didelegasikan ke *cloud backend* (Kaggle), menggunakan Firebase sebagai *Message Broker*, dan disajikan melalui *cloud hosting* Vercel (PaaS). Pendekatan *microservices* ini selaras dengan prinsip Dragoni et al. yang menyatakan bahwa *microservices* memungkinkan pengembangan, deployment, dan scaling komponen sistem secara independen [2].

1.2 Tujuan

- Menyediakan prediksi harga properti secara real-time berbasis model XGBoost yang terlatih di cloud.
- Mengimplementasikan arsitektur *Event-Driven Microservices* yang memisahkan frontend, message broker, dan backend AI.
- Memanfaatkan infrastruktur cloud (Kaggle, Firebase, Vercel) untuk mengatasi keterbatasan komputasi lokal.
- Menerapkan praktik terbaik MLOps dalam deployment model kecerdasan buatan yang efisien dan skalabel.

BAB II

PENGUMPULAN DATA

2.1 Sumber dan Karakteristik Data

Data yang digunakan merupakan data tabular dari dataset publik di platform Kaggle, yaitu “House Prices – Advanced Regression Techniques”. Dataset ini merupakan *benchmark* standar dalam komunitas *machine learning* untuk tugas regresi harga properti dan tersedia secara terbuka untuk keperluan akademis.

Karakteristik dataset:

- Data terdiri dari puluhan kolom yang mendeskripsikan fitur-fitur fisik rumah (luas tanah, jumlah kamar, luas garasi, kualitas bahan bangunan, dll.) dan satu kolom target yaitu SalePrice (Harga Jual).
- Format data: CSV (Comma-Separated Values) dengan tipe data numerik dan kategorikal.

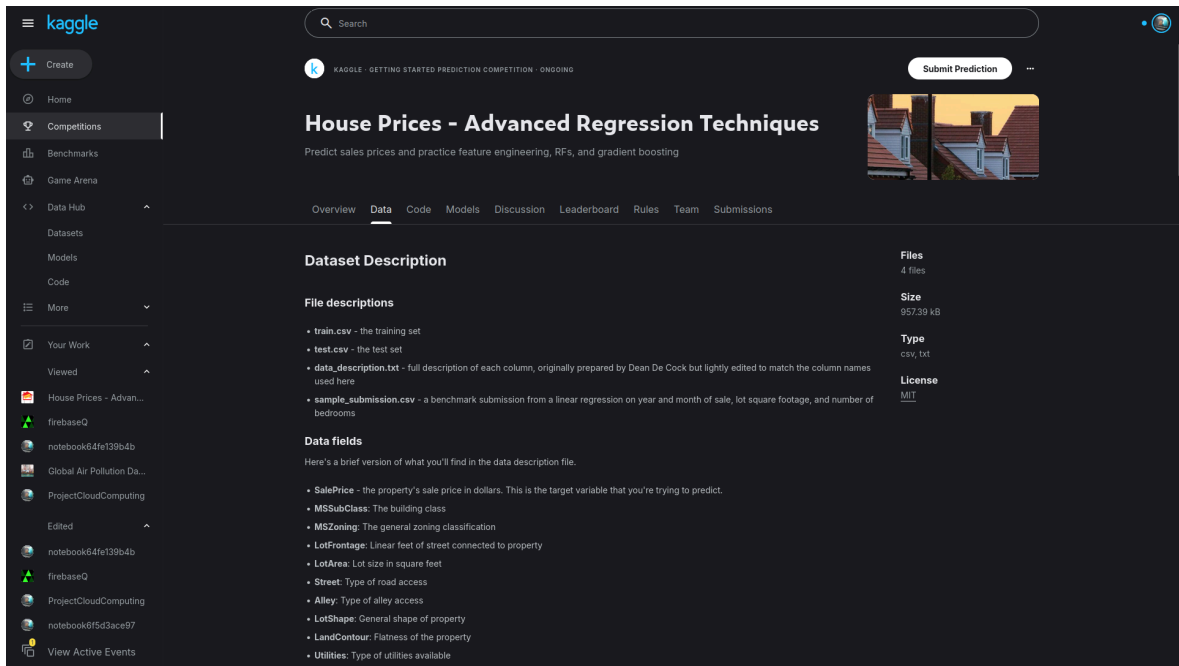
2.2 Pemisahan Data (Train & Test)

Sesuai standar *Machine Learning Operations* (MLOps), data dibagi menjadi dua file terpisah:

- train.csv – digunakan untuk melatih model AI dan melakukan validasi silang (cross-validation).
- test.csv – berfungsi sebagai data buta (unseen data) untuk menguji model setelah proses training selesai.

2.3 Tampilan Dataset di Platform Kaggle

Dataset “House Prices – Advanced Regression Techniques” diakses langsung dari platform Kaggle. Gambar 1 menunjukkan halaman dataset Kaggle yang memuat file train.csv dan test.csv beserta informasi deskriptif variabel-variabel yang tersedia.



Gambar 1. Halaman dataset House Prices pada platform Kaggle

Pada halaman dataset tersebut terlihat metadata dataset, daftar file (train.csv, test.csv, data_description.txt), serta jumlah kolom dan baris data. Dataset train.csv memuat 1.460 baris data dengan 81 kolom fitur, sedangkan test.csv memuat 1.459 baris tanpa kolom target SalePrice.

BAB III

IMPLEMENTASI CLOUD COMPUTING DAN ANALISIS DATA

3.1 Model Layanan Cloud yang Digunakan

Sistem ini mengimplementasikan tiga layanan cloud secara bersamaan. Berdasarkan klasifikasi Erl et al. [1], sistem memanfaatkan lapisan IaaS, PaaS, dan BaaS secara sinergis:

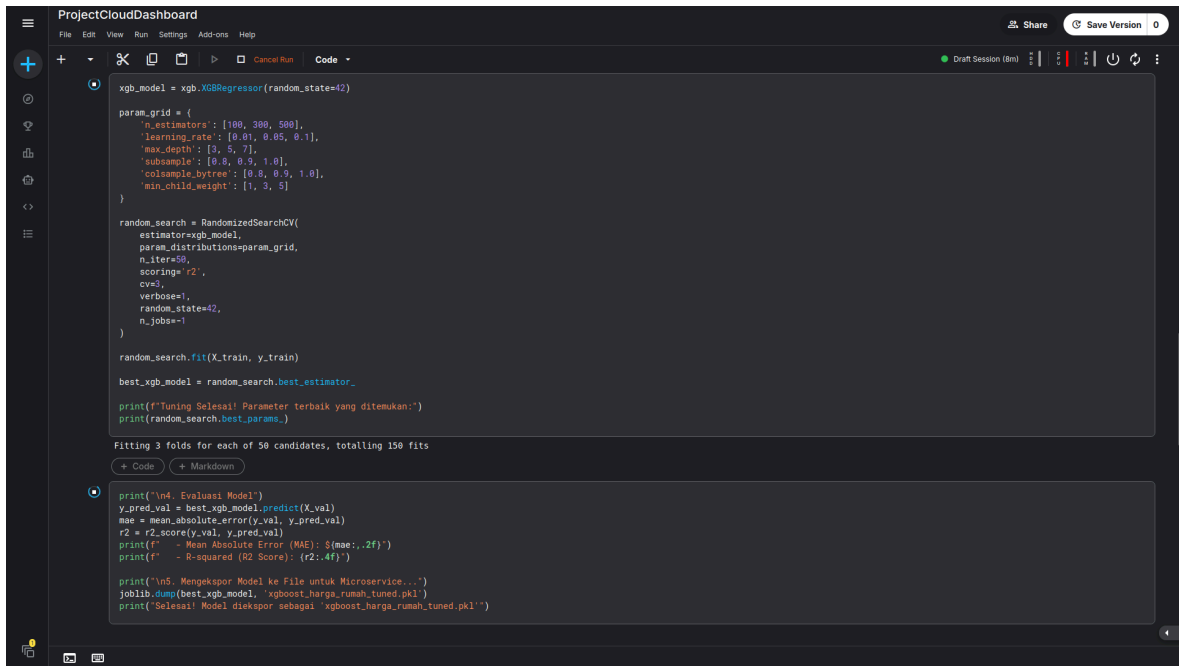
Tabel 1. Komponen Layanan Cloud pada Sistem

Komponen Sistem	Layanan Cloud / Model	Peran dan Fungsi
Frontend Presentation Layer	Vercel (PaaS)	Menghosting antarmuka pengguna berbasis HTML/JavaScript statis. Ringan tanpa beban komputasi AI.
Data Broker & Message Queue	Firebase Realtime Database (BaaS)	Jembatan komunikasi real-time antara Vercel dan Kaggle. Mengelola antrean request dengan status pending dan done.
Backend AI Worker Core	Kaggle Notebook (IaaS)	Menjalankan skrip Python secara kontinu untuk memuat model XGBoost, memantau database, dan memproses prediksi.

Pemilihan arsitektur tiga layanan ini didasarkan pada pertimbangan teknis dan ekonomis: Kaggle menyediakan CPU/RAM hingga 16 GB secara gratis; Firebase menyediakan komunikasi asinkron berbasis event tanpa REST API server terpisah; dan Vercel menyediakan CDN global dengan latensi akses yang rendah. Pendekatan *serverless* ini senada dengan temuan Kaur et al. yang menyatakan bahwa *serverless computing* menawarkan skalabilitas otomatis dan pengurangan overhead manajemen infrastruktur yang signifikan [3].

3.2 Tampilan Lingkungan Kaggle Notebook

Proses training model XGBoost dan Hyperparameter Tuning dilakukan di dalam Kaggle Notebook. Gambar 2 menunjukkan tampilan Kaggle Notebook yang sedang menjalankan skrip Python untuk proses training, tuning, dan ekspor model.



```
xgb_model = xgb.XGBRegressor(random_state=42)

param_grid = {
    'n_estimators': [100, 300, 500],
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [3, 5, 7],
    'subsample': [0.8, 0.9, 1.0],
    'colsample_bytree': [0.8, 0.9, 1.0],
    'min_child_weight': [1, 3, 5]
}

random_search = RandomizedSearchCV(
    estimator=xgb_model,
    param_distributions=param_grid,
    n_iter=50,
    scoring='r2',
    cv=3,
    verbose=1,
    random_state=42,
    n_jobs=-1
)

random_search.fit(X_train, y_train)

best_xgb_model = random_search.best_estimator_

print(f'Tuning Selesai! Parameter terbaik yang ditemukan:')
print(random_search.best_params_)

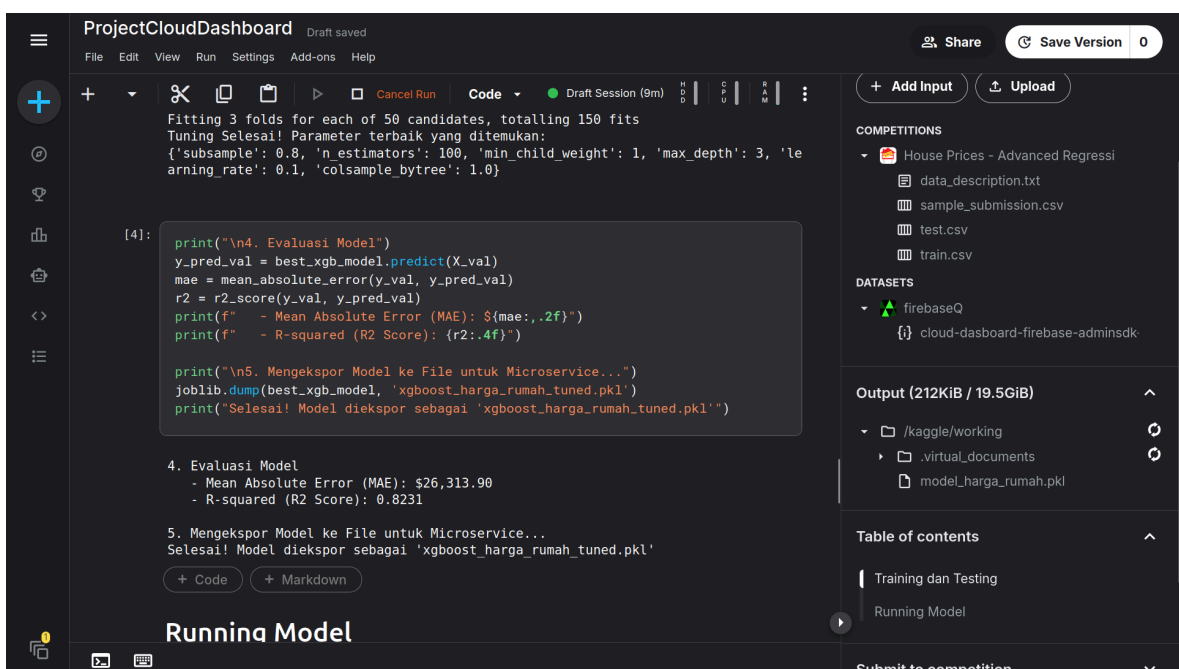
Fitting 3 folds for each of 50 candidates, totalling 150 fits

print("\n4. Evaluasi Model")
y_pred_val = best_xgb_model.predict(X_val)
mae = mean_absolute_error(y_val, y_pred_val)
r2 = r2_score(y_val, y_pred_val)
print(f" - Mean Absolute Error (MAE): ${mae:.2f}")
print(f" - R-squared (R2 Score): {r2:.4f}")

print("\n5. Mengekspor Model ke File untuk Microservice...")
joblib.dump(best_xgb_model, 'xgboost_harga_rumah_tuned.pkl')
print("Selesai! Model diekspor sebagai 'xgboost_harga_rumah_tuned.pkl'")
```

Gambar 2. Tampilan Kaggle Notebook saat menjalankan proses training model XGBoost

Pada Gambar 2 terlihat sel-sel kode Python yang meliputi: (1) import library (xgboost, sklearn, pandas, firebase_admin), (2) proses pembersihan data dan imputasi missing values, (3) eksekusi RandomizedSearchCV dengan 3-fold cross-validation, serta (4) output metrik evaluasi berupa nilai R² Score dan MAE. Kaggle Notebook memanfaatkan sumber daya cloud (CPU 4-core, RAM 16 GB) yang jauh melampaui kapasitas komputasi lokal standar.



```
Fitting 3 folds for each of 50 candidates, totalling 150 fits
Tuning Selesai! Parameter terbaik yang ditemukan:
{'subsample': 0.8, 'n_estimators': 100, 'min_child_weight': 1, 'max_depth': 3, 'learning_rate': 0.1, 'colsample_bytree': 1.0}
```

```
[4]: print("\n4. Evaluasi Model")
y_pred_val = best_xgb_model.predict(X_val)
mae = mean_absolute_error(y_val, y_pred_val)
r2 = r2_score(y_val, y_pred_val)
print(f" - Mean Absolute Error (MAE): ${mae:.2f}")
print(f" - R-squared (R2 Score): {r2:.4f}")

print("\n5. Mengekspor Model ke File untuk Microservice...")
joblib.dump(best_xgb_model, 'xgboost_harga_rumah_tuned.pkl')
print("Selesai! Model diekspor sebagai 'xgboost_harga_rumah_tuned.pkl'")
```

```
4. Evaluasi Model
- Mean Absolute Error (MAE): $26,313.90
- R-squared (R2 Score): 0.8231

5. Mengekspor Model ke File untuk Microservice...
Selesai! Model diekspor sebagai 'xgboost_harga_rumah_tuned.pkl'
```

Runnina Model

COMPETITIONS

- House Prices - Advanced Regressi
 - data_description.txt
 - sample_submission.csv
 - test.csv
 - train.csv

DATASETS

- firebaseQ
 - cloud-dasboard-firebase-adminsdk

Output (212KiB / 19.5GiB)

- /kaggle/working
 - virtual_documents
 - model_harga_rumah.pkl

Table of contents

- Training dan Testing
- Running Model

Submit to competition

Gambar 3. Output evaluasi model dan konfirmasi ekspor file model.pkl pada Kaggle Notebook

Gambar 3 menampilkan hasil akhir proses training: nilai metrik evaluasi (R^2 dan MAE) dari model XGBoost terbaik hasil Hyperparameter Tuning, serta konfirmasi bahwa file *model.pkl* berhasil disimpan ke Kaggle Dataset. File model inilah yang kemudian dimuat oleh AI Worker saat menjalankan inferensi prediksi harga properti.

3.3 Tahapan Analisis Data

3.3.1 Pembersihan Data (*Data Cleaning*)

Fitur yang dipilih dibatasi pada empat variabel paling esensial: Luas Bangunan (GrLivArea), Jumlah Kamar (BedroomAbvGr), Kamar Mandi (FullBath), dan Garasi (GarageCars). Pembatasan fitur dilakukan secara intentional untuk memastikan model generalisasi baik pada input pengguna yang minimal. Nilai kosong (*missing values*) ditangani dengan teknik imputasi (diisi nilai 0) agar tidak merusak proses training.

3.3.2 Pemilihan Algoritma: XGBoost

Sistem menggunakan XGBoost (*Extreme Gradient Boosting*) alih-alih regresi linier dasar. Chen dan Guestrin mendefinisikan XGBoost sebagai sistem boosting pohon yang dapat diskalakan, mengimplementasikan algoritma gradient boosting dengan optimasi sistem canggih, termasuk komputasi paralel dan pemangkasan pohon berbasis regularisasi [4].

XGBoost dipilih karena:

- Kemampuan memetakan hubungan non-linear pada data tabular harga rumah.
- Efisiensi komputasi tinggi melalui pemrosesan paralel.
- Ketahanan terhadap overfitting melalui regularisasi L1 dan L2 bawaan.
- Performa superior dibandingkan algoritma regresi konvensional pada dataset tabular [4].

3.3.3 Hyperparameter Tuning di Cloud

Untuk memastikan akurasi tertinggi, diterapkan teknik optimasi RandomizedSearchCV dengan 3-fold *Cross-Validation*. Kaggle Kernel – dengan akses CPU/RAM besar secara gratis – mencari kombinasi parameter terbaik secara acak, meliputi: *learning_rate*, *max_depth*, *n_estimators*, *subsample*, dan *colsample_bytree*. Proses ini merupakan pemanfaatan nyata infrastruktur cloud untuk komputasi intensif yang tidak efisien jika dilakukan di perangkat lokal.

3.3.4 Evaluasi Model

Setelah komputasi di cloud selesai, model dievaluasi menggunakan dua metrik utama:

- R-squared (R^2 Score): Mengukur proporsi variansi data yang dapat dijelaskan oleh model. Semakin mendekati 1,0, semakin baik.
- Mean Absolute Error (MAE): Mengukur rata-rata deviasi absolut prediksi dari nilai aktual dalam satuan dolar.

Model terbaik (*Best Estimator*) kemudian diekspor dalam format .pkl menggunakan library pickle Python untuk tahap deployment ke infrastruktur cloud.

3.4 Implementasi Arsitektur Event-Driven Microservices

Arsitektur *event-driven* yang diterapkan mengimplementasikan pola komunikasi asinkron berbasis peristiwa. Dragoni et al. mendefinisikan microservices sebagai paradigma arsitektur di mana aplikasi dikembangkan sebagai sekumpulan layanan kecil yang berjalan mandiri, berkomunikasi melalui mekanisme ringan, dan dapat di-deploy secara independen [2].

Alur kerja sistem (Event-Driven Flow):

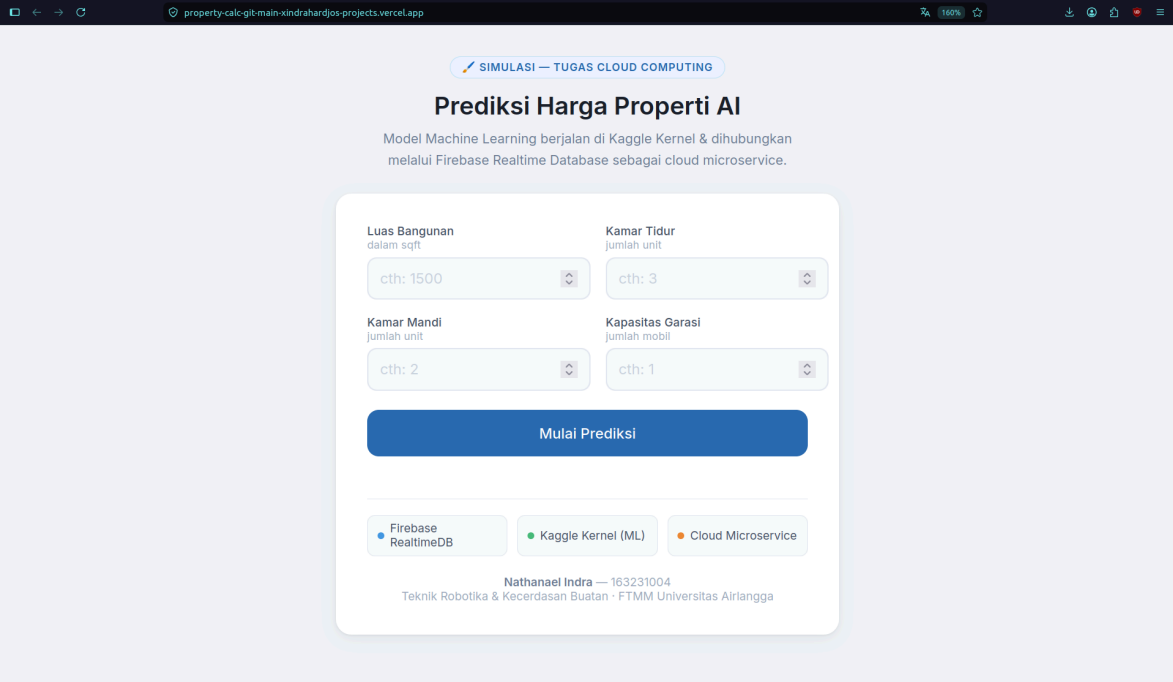
1. Input Pengguna: Pengguna memasukkan parameter properti melalui antarmuka web di Vercel.
2. Publish Event: Frontend Vercel mempublikasikan payload JSON ke Firebase Realtime Database dengan status pending.
3. Event Detection: AI Worker di Kaggle Notebook memantau perubahan data (event listener) di Firebase dan mengambil request baru.
4. Inferensi AI: Worker memuat model XGBoost (.pkl) ke memori dan menjalankan inferensi prediksi harga.
5. Publish Result: Hasil prediksi ditulis kembali ke Firebase dengan status done.
6. Display Result: Frontend Vercel mendeteksi perubahan status dan menampilkan hasil prediksi kepada pengguna.

BAB IV

PEMBAHASAN DAN PENGUJIAN SISTEM

4.1 Tampilan Dashboard Antarmuka Pengguna

Antarmuka pengguna (dashboard) dikembangkan menggunakan HTML dan JavaScript statis yang dihosting di Vercel. Dashboard menyediakan form input sederhana untuk memasukkan empat parameter properti: Luas Bangunan (GrLivArea), Jumlah Kamar (BedroomAbvGr), Kamar Mandi (FullBath), dan Garasi (GarageCars). Gambar 4 menunjukkan tampilan dashboard sebelum pengguna melakukan input.

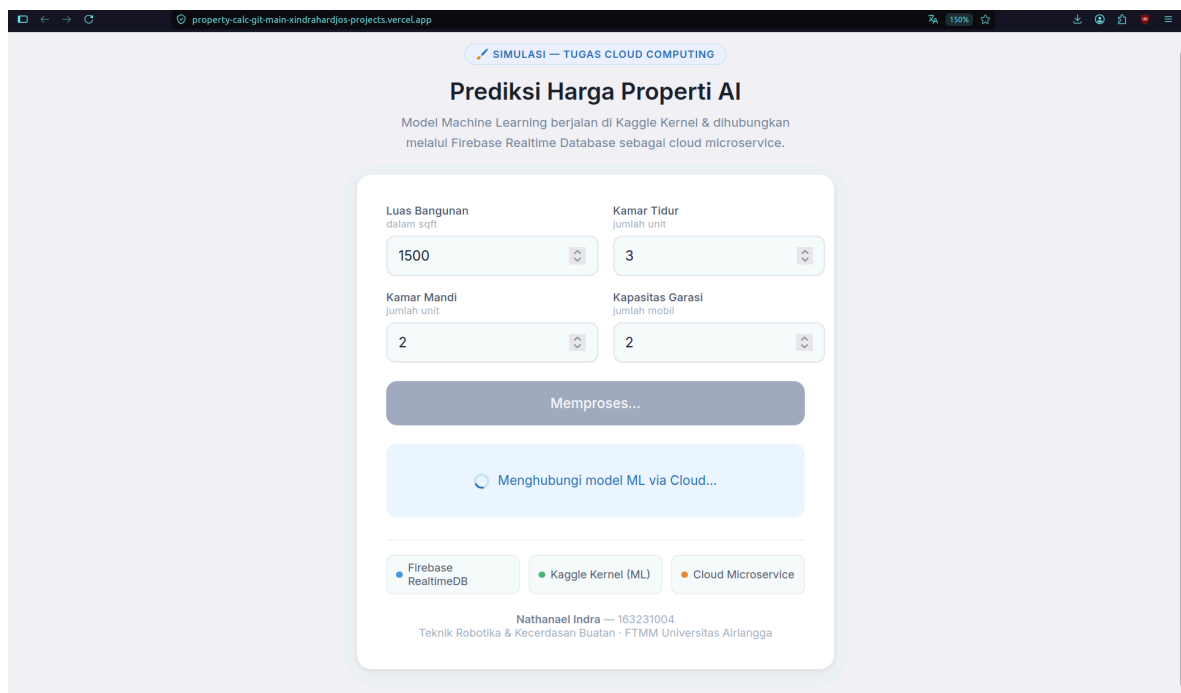


Gambar 4. Tampilan awal dashboard Kalkulator Prediksi Harga Properti AI di Vercel

Dashboard dirancang minimalis agar dapat diakses dari perangkat apapun dengan koneksi internet, sesuai dengan prinsip *broad network access* pada cloud computing [1]. Seluruh proses inferensi AI tidak berjalan di sisi klien, sehingga dashboard dapat diakses bahkan dari perangkat dengan spesifikasi rendah.

4.2 Simulasi Pengujian – Skenario Input dan Prediksi

Pengujian dilakukan dengan memasukkan parameter properti pada dashboard dan mengamati respons sistem secara end-to-end, mulai dari perubahan data di Firebase hingga munculnya hasil prediksi pada dashboard. Gambar 5 menunjukkan dashboard setelah pengguna mengisi form dan menekan tombol prediksi.

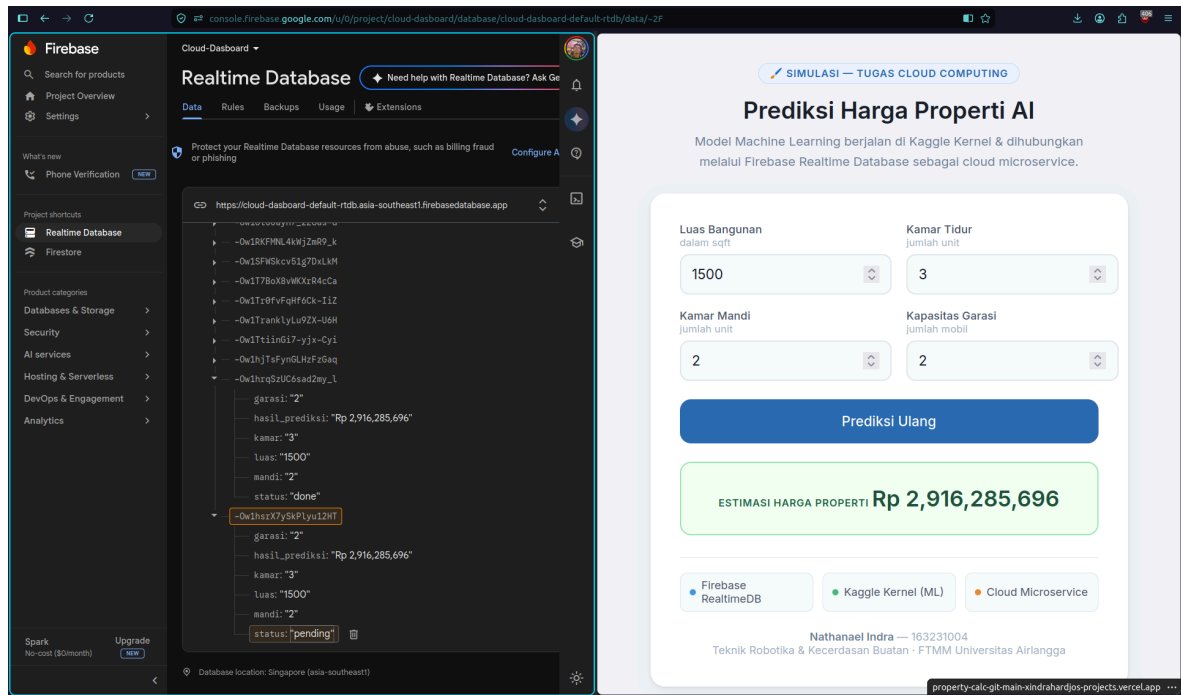


Gambar 5. Dashboard setelah pengguna mengisi parameter dan mengirimkan request prediksi

Pada contoh pengujian di Gambar 5, pengguna memasukkan nilai: GrLivArea = 1.500 ft², BedroomAbvGr = 3, FullBath = 2, dan GarageCars = 2. Setelah tombol prediksi ditekan, frontend Vercel secara langsung mempublikasikan payload JSON tersebut ke Firebase Realtime Database dengan status *pending*, tanpa melakukan komputasi apapun di sisi browser.

4.3 Perubahan Data pada Firebase Realtime Database

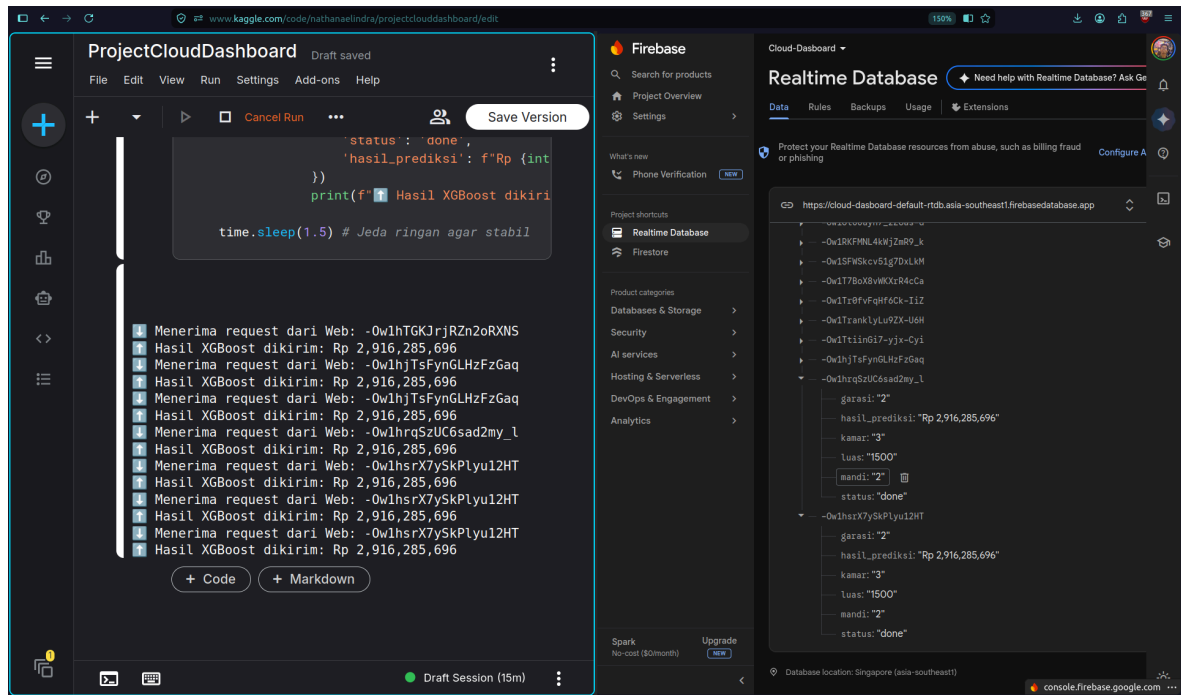
Firebase Realtime Database berfungsi sebagai message broker yang menjembatani komunikasi antara frontend (Vercel) dan backend AI (Kaggle). Gambar 6 menunjukkan kondisi Firebase Console sesaat setelah request prediksi dikirimkan oleh frontend, yaitu saat data masih berstatus *pending*.



Gambar 6. Firebase Realtime Database menampilkan request dengan status pending

Pada Gambar 6 terlihat node `/requests/{id}` di Firebase Console memuat payload JSON yang dikirimkan oleh frontend: nilai keempat parameter properti serta field status bernilai *"pending"*. Kondisi ini merupakan sinyal bagi AI Worker di Kaggle untuk segera mengambil dan memproses data tersebut.

Gambar 7 menunjukkan kondisi Firebase setelah AI Worker di Kaggle selesai memproses prediksi dan menuliskan hasilnya kembali ke database, yaitu saat status berubah menjadi *done* dan field *result* terisi dengan nilai prediksi harga.

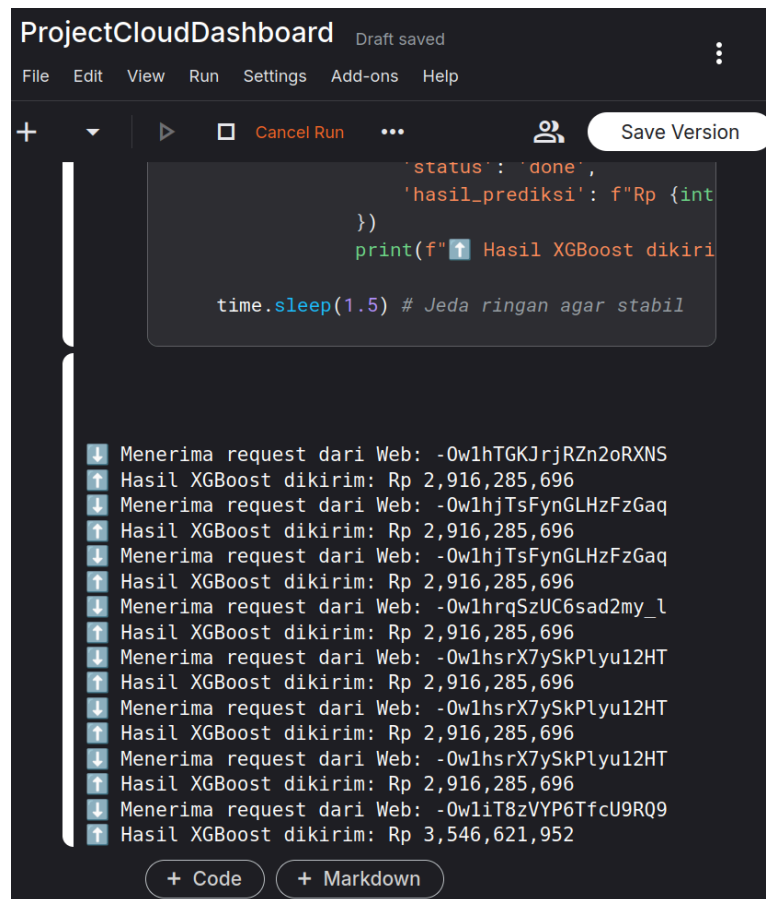


Gambar 7. Firebase Realtime Database setelah AI Worker menuliskan hasil prediksi (status: done)

Gambar 7 memperlihatkan perubahan signifikan pada node Firebase: field status telah berubah dari “pending” menjadi “done”, dan field *result* kini memuat nilai prediksi harga properti dalam satuan dolar Amerika Serikat. Perubahan ini terdeteksi secara real-time oleh event listener pada frontend Vercel, yang kemudian menampilkan hasil prediksi kepada pengguna.

4.4 Perubahan pada Kaggle Notebook saat Memproses Request

AI Worker yang berjalan di Kaggle Notebook secara kontinu memantau perubahan data di Firebase menggunakan mekanisme *event listener*. Ketika request baru dengan status *pending* terdeteksi, Worker langsung mengambil data, menjalankan inferensi model XGBoost, dan menulis hasil kembali ke Firebase. Gambar 8 menunjukkan tampilan Kaggle Notebook saat Worker aktif mendeteksi dan memproses request.



```
ProjectCloudDashboard Draft saved
File Edit View Run Settings Add-ons Help

+ [Run] [Cancel Run] ... [Save Version]

    'status': 'done',
    'hasil_prediksi': f"Rp {int
    })
    print(f"⬆ Hasil XGBoost dikiri
    time.sleep(1.5) # Jeda ringan agar stabil

⬇ Menerima request dari Web: -0w1hTGKJrjRZn2oRXNS
⬆ Hasil XGBoost dikirim: Rp 2,916,285,696
⬇ Menerima request dari Web: -0w1hjTsFynGLHzFzGaq
⬆ Hasil XGBoost dikirim: Rp 2,916,285,696
⬇ Menerima request dari Web: -0w1hjTsFynGLHzFzGaq
⬆ Hasil XGBoost dikirim: Rp 2,916,285,696
⬇ Menerima request dari Web: -0w1hrqSzUC6sad2my_l
⬆ Hasil XGBoost dikirim: Rp 2,916,285,696
⬇ Menerima request dari Web: -0w1hsrX7ySkPlyu12HT
⬆ Hasil XGBoost dikirim: Rp 2,916,285,696
⬇ Menerima request dari Web: -0w1hsrX7ySkPlyu12HT
⬆ Hasil XGBoost dikirim: Rp 2,916,285,696
⬇ Menerima request dari Web: -0w1hsrX7ySkPlyu12HT
⬆ Hasil XGBoost dikirim: Rp 2,916,285,696
⬇ Menerima request dari Web: -0w1iT8zVYP6TfcU9RQ9
⬆ Hasil XGBoost dikirim: Rp 3,546,621,952

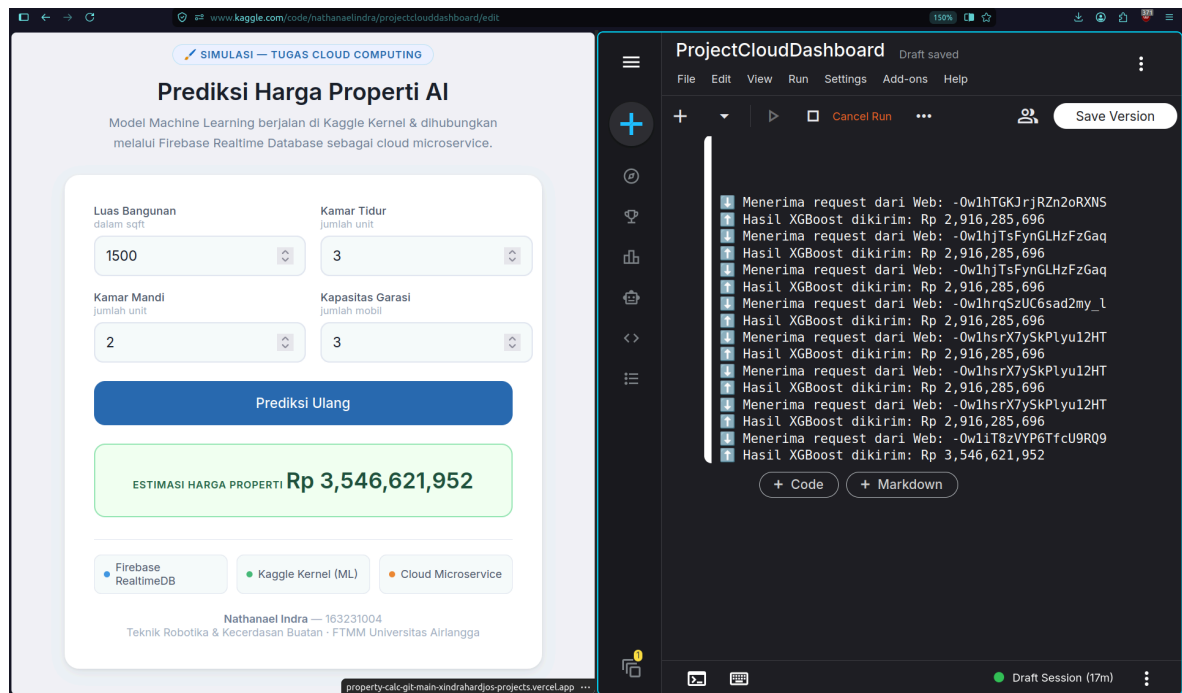
+ Code + Markdown
```

Gambar 8. *Kaggle Notebook menampilkan log AI Worker saat mendeteksi dan memproses request prediksi*

Pada Gambar 8 terlihat output log dari AI Worker yang menunjukkan: (1) deteksi perubahan data di Firebase, (2) ekstraksi nilai keempat fitur input, (3) proses pemuatan model XGBoost dari file .pkl, (4) eksekusi inferensi prediksi, dan (5) konfirmasi penulisan hasil prediksi kembali ke Firebase. Seluruh proses ini berjalan dalam hitungan detik berkat efisiensi komputasi cloud Kaggle.

4.5 Hasil Prediksi pada Dashboard

Setelah AI Worker menuliskan hasil prediksi ke Firebase dan mengubah status menjadi *done*, event listener pada frontend Vercel mendeteksi perubahan tersebut dan secara otomatis menampilkan nilai prediksi harga kepada pengguna. Gambar 9 menunjukkan tampilan akhir dashboard setelah hasil prediksi berhasil dimuat.



Gambar 9. Tampilan dashboard setelah hasil prediksi harga properti berhasil dimuat dari Firebase

Gambar 9 memperlihatkan hasil prediksi harga properti yang ditampilkan pada dashboard. Nilai prediksi muncul secara otomatis tanpa perlu me-refresh halaman, berkat mekanisme *real-time listener* yang terhubung langsung ke Firebase Realtime Database. Keseluruhan siklus – dari input pengguna hingga tampilnya hasil prediksi – berlangsung dalam waktu beberapa detik, membuktikan efektivitas arsitektur Event-Driven Microservices yang diterapkan.

4.6 Efisiensi dan Efektivitas Penggunaan Cloud

4.6.1 Decoupling Fungsional

Frontend (Vercel) dan Backend AI (Kaggle) dipisahkan secara total melalui Firebase sebagai message broker. Prinsip decoupling ini merupakan inti dari arsitektur microservices sebagaimana dijabarkan Dragoni et al. [2]: setiap layanan memiliki siklus hidup yang independen. Pembaruan model ML di Kaggle tidak mengakibatkan downtime pada antarmuka pengguna di Vercel.

4.6.2 Efisiensi Biaya dan Sumber Daya

Proses Hyperparameter Tuning yang intensif dilakukan sepenuhnya menggunakan infrastruktur Kaggle Kernel secara gratis. Pendekatan ini sesuai dengan model serverless yang dikaji Kaur et al. [3], di mana biaya operasional hanya dikenakan berdasarkan sumber daya yang benar-benar digunakan.

4.6.3 Skalabilitas Horizontal

Arsitektur yang diterapkan memungkinkan skalabilitas horizontal: jika volume request meningkat, dapat ditambahkan lebih banyak Kaggle Worker yang memantau Firebase secara paralel tanpa mengubah kode frontend. Prinsip elastisitas ini merupakan karakteristik fundamental cloud computing menurut Erl et al. [1].

4.7 Kelebihan dan Kekurangan Sistem

Kelebihan

- Arsitektur microservices memberikan fleksibilitas tinggi dalam pengembangan dan pembaruan komponen secara independen.
- Biaya operasional mendekati nol berkat penggunaan tier gratis dari Kaggle, Firebase, dan Vercel.
- Model XGBoost menghasilkan prediksi akurat berkat optimasi Hyperparameter Tuning di cloud.

Kekurangan

- Kaggle Worker membutuhkan sesi aktif – tidak berjalan otomatis 24/7 tanpa intervensi manual.
- Latensi prediksi bergantung pada koneksi Firebase dengan Kaggle dan dapat bervariasi.
- Fitur prediksi dibatasi empat variabel utama; akurasi dapat ditingkatkan dengan lebih banyak fitur.

BAB V

KESIMPULAN

Sistem “Kalkulator Cerdas Prediksi Harga Properti AI” berhasil dikembangkan menggunakan XGBoost yang dipadukan dengan ekosistem cloud computing modern. Melalui arsitektur Event-Driven Microservices, proses Hyperparameter Tuning dan inferensi model dapat berjalan dengan efektivitas dan skalabilitas yang tinggi [2].

Hasil pengujian sistem secara *end-to-end* yang dibuktikan melalui dokumentasi screenshot pada Gambar 4 hingga 9. Hal menunjukkan bahwa alur komunikasi Event-Driven antara dashboard Vercel, Firebase Realtime Database, dan AI Worker Kaggle berjalan dengan baik. Perubahan status data di Firebase dari *pending* menjadi *done* berlangsung dalam hitungan detik, membuktikan efektivitas arsitektur yang diimplementasikan.

Algoritma XGBoost [4] terbukti superior dalam menangani data tabular harga properti. Firebase Realtime Database terbukti handal bertindak sebagai broker komunikasi real-time yang menghubungkan antarmuka front-end ringan di Vercel dengan worker pemrosesan berat di Kaggle. Pendekatan serverless dan event-driven yang diterapkan, memungkinkan operasional sistem dengan biaya infrastruktur mendekati nol [1, 3]. Sistem ini merepresentasikan praktik MLOps dalam mendeploy model kecerdasan buatan secara efektif, efisien, dan skalabel.

DAFTAR PUSTAKA

- [1]T. Erl, Z. Mahmood, and R. Puttini, Cloud Computing: Concepts, Technology & Architecture. Upper Saddle River, NJ: Prentice Hall, 2013.
- [2]N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, “Microservices: yesterday, today, and tomorrow,” in Present and Ulterior Software Engineering, M. Mazzara and B. Meyer, Eds. Cham: Springer, 2017, pp. 195–216.
- [3]K. Kaur, M. Günther, A. Sasan, and A. Balador, “Serverless computing: A survey of opportunities, challenges and applications,” J. Cloud Comput., vol. 10, no. 1, pp. 1–42, 2021, doi: 10.1186/s13677-021-00234-4.
- [4]T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, New York, NY: ACM, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.